Distributed LLM Inference and Parallelism

pig7selene

September 5, 2026

Abstract

A serving deployment needs more than one accelerator when a model and its runtime state exceed one device's memory, or when one device cannot meet the workload's latency and throughput objectives. This chapter distinguishes replication of complete inference groups from parallel execution of one request, then develops Tensor, Pipeline, Sequence, and Expert Parallelism in the inference critical path. It treats collective communication, Prefill–Decode disaggregation, and multidimensional placement as systems choices whose value depends on memory, topology, workload shape, and service objectives.

1 Introduction

The preceding inference chapters considered the execution of a request, the KV Cache that accompanies it, the scheduler that shares a device, and an algorithm that reduces serial Decode iterations. Those optimizations do not remove two fundamental deployment limits. First, model weights, temporary workspaces, and the active KV Cache may not fit in a single accelerator's usable memory. Second, even a model that fits may leave one accelerator unable to satisfy the arrival rate or latency objectives of a production workload. Distributed inference distributes capacity, computation, or both across coordinated accelerators.

The same words used for distributed training appear here, but their operational meaning changes. Chapter 10 derives how ranks synchronize gradients and optimizer updates to preserve one training objective. In serving, parameters are normally fixed; there is no backward pass, optimizer state, or gradient all-reduce. Instead, a request must carry a coherent prefix, positional state, and KV Cache through a sequence of forward computations, often while a scheduler balances many independent requests. The critical question is not whether ranks agree on an update, but whether their placement and communication preserve the logits of the intended model while meeting a latency and capacity target.

Let n_R denote the number of request-serving replicas, n_T the Tensor Parallel degree within one execution group, and n_P its Pipeline Parallel degree. If no other independent dimensions are present, the deployed accelerator count is

$$W = n_R n_T n_P. \tag{1}$$

An **execution group** is the set of ranks that jointly runs one model forward pass under a chosen model-parallel plan. A replica is a complete copy of such a group, not merely one device. An Expert Parallel degree may add another factor in a Mixture-of-Experts (MoE) design, while Sequence Parallelism commonly reuses an existing Tensor Parallel group rather than introducing a wholly independent factor. This accounting is a placement description, not a prescription: the useful degrees are those justified by the model, network, and request distribution.

2 Capacity, Throughput, and Inference Semantics

Inference parallelism begins with two distinct questions. **Capacity parallelism** asks how to run one model execution when its weights or request state cannot reside on one accelerator. **Throughput parallelism** asks how many independent request executions can proceed at once. Model-parallel techniques answer the first question by splitting one execution group. Replication answers the second by creating multiple complete execution groups. A practical service commonly needs both: each

replica may itself use Tensor or Pipeline Parallelism, while a front-end routes different requests to different replicas.

This distinction prevents a misleading analogy with training. During Data Parallel training, every replica evaluates different examples and later reconciles gradients before advancing the same parameters. A serving replica simply owns a different set of requests. It can decode those requests independently because their prefixes and caches are distinct, and no update must be reconciled after a token is emitted. Inference routing must nevertheless preserve request affinity: after a Prefill, later Decode steps must reach the ranks that own the corresponding KV state, unless the system explicitly transfers or reconstructs that state.

Inference also exposes phase asymmetry. As Chapter 19 explains, Prefill processes many prompt positions and can exploit comparatively large matrix operations; Decode performs a small incremental step for each generated token. Thus a parallel plan that gives excellent Prefill throughput can still harm interactive Decode latency if it inserts a collective or inter-stage transfer at every token. Pope et al. analyze multidimensional partitioning for Transformer inference under this latency–throughput tension [1]. The relevant unit of evaluation is a realistic mixture of prompt lengths, output lengths, arrival rates, and service-level objectives, not a nominal division of FLOPs by W.

3 Tensor Parallelism in the Inference Critical Path

Tensor Parallelism splits the arithmetic of one Transformer layer over n_T cooperating ranks. Its algebra is the same as in Chapter 10, but at inference time the corresponding collective lies directly on the route to the next-token logits. Consider a linear map $Y = XW$, where $X \in \mathbb{R}^{b \times d_{\text{in}}}$ and $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. For a column partition,

$$W = \left[W^1, \ldots, W^{n_T}\right], \quad Y = \operatorname{concat}\left(XW^1, \ldots, XW^{n_T}\right). \tag{2}$$

Each rank owns an output-feature slice and can compute its local product. A following operation may consume the slices directly, or it may require an all-gather to materialize the full feature dimension. For a row partition, split $X = \left[X^1, \ldots, X^{n_T}\right]$ and partition W by its rows. Each rank forms a partial output, and the complete result is

$$Y = \sum_{j=1}^{n_T} X^j W^j. \tag{3}$$

This sum generally requires an all-reduce. A Transformer MLP can arrange its expanding projection as a column-parallel operation and its contracting projection as a row-parallel operation, so that the activation remains local between them. Attention heads can similarly be placed on different ranks before an output projection reconciles the result. This arrangement is the intra-layer model-parallel pattern developed in Megatron-LM [2].

The saving is substantial when one rank could not otherwise hold the layer weights or execute the layer at the needed scale. The cost is recurrent communication. A Decode step visits every Transformer layer, so even a small synchronization repeated at every layer can set the token latency. Tensor Parallel groups are consequently often placed within a high-bandwidth, low-latency locality domain, such as accelerators linked by a fast intra-node interconnect. Crossing a slower network for every layer is possible, but it can turn communication latency into the dominant Decode cost. Group size is therefore a systems decision: a larger n_T lowers local weight memory while increasing collective frequency and potentially making each local matrix multiplication too small to use hardware efficiently.

4 Pipeline Parallelism and Request Replication

Pipeline Parallelism partitions model depth. If there are n_P stages, stage j stores a consecutive range of Transformer layers and sends its output activation to stage $j + 1$. During inference, this transfer is principally point-to-point communication: an activation travels forward through the layer stages,

then the final stage produces or makes available the next-token logits. A request's Decode loop repeats that stage chain for every emitted token.

Microbatches improve pipeline occupancy when a group is serving many independent request sequences. However, the training bubble formula in Chapter 10 should not be transferred mechanically. Training schedules use both forward and backward waves and can amortize a fixed optimizer-step batch. Interactive inference has no backward wave and often cares about the latency of one request's next token. A single request still traverses every stage serially, so Pipeline Parallelism can solve a capacity problem while adding hop latency. The benefit rises when concurrent Prefill work or many active Decode sequences provide enough independent work to keep stages occupied; the cost rises with imbalanced stages, short batches, and long inter-stage links. GPipe established the layer-stage abstraction, while large language-model systems combine it with Tensor Parallelism at scale [3], [4].

Replica parallelism is different. A service can make n_R copies of a full execution group, each possibly composed from n_T Tensor Parallel ranks and n_P Pipeline stages. The router assigns a new request to one replica, and that replica retains the request's KV Cache through Decode. Replicas increase aggregate request capacity and can isolate work queues, but they do not make a model fit if each replica is still too small. Conversely, adding more Tensor Parallel ranks to a single group does not create independent request capacity in the same way; the ranks cooperate on the same forward pass and must synchronize.

The following figure records this ownership boundary. It is deliberately logical rather than tied to a serving framework.

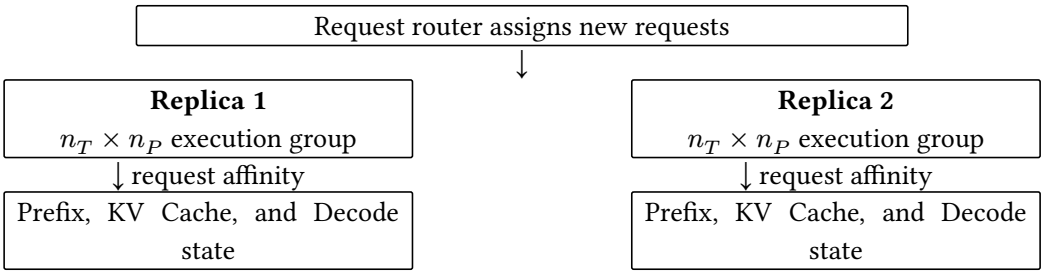

Figure 1: Request replication creates independent complete execution groups. Model parallel ranks within one group cooperate on one request; a request must remain with its cache-owning group unless the service performs an explicit state handoff.

Figure 1 separates **routing** from **execution**. A router can use queue length, expected resource demand, priority, and replica health to choose an entry point, but it must not silently treat a request's KV state as globally available. The scheduling policies in Chapter 22 act within and across these ownership domains.

5 Sequence and Expert Parallelism

Sequence Parallelism partitions compatible activation work across ranks, often along the sequence dimension and often in conjunction with a Tensor Parallel group. Its precise implementation varies: some operations retain local token slices, while others all-gather a representation before a layer and reduce-scatter it afterward. For inference, the opportunity depends on the current phase. Prefill has many prompt positions and can expose larger sequence-axis work; a one-token Decode step offers little sequence dimension to divide for a single request. Sequence Parallelism can therefore reduce replicated activation or communication pressure in an appropriate layout, but it is not a universal way to accelerate Decode. The important contract is to specify the shard axis and the gather and scatter boundaries, as Chapter 10 emphasizes.

MoE models introduce **Expert Parallelism**. In an MoE layer, a router maps token representations to a small subset of experts. If experts reside on different ranks, tokens must be dispatched to their selected expert owners, evaluated, and returned to the position that combines their outputs. Let h_i

be the representation of token i, let E be the expert set, and let $\mathcal{E}_i \subset E$ be the selected experts. A schematic MoE output is

$$f_{\mathrm{MoE}(h_i)} = \sum_{e \in \mathcal{E}_i} g_{i,e} f_{e(h_i)}, \tag{4}$$

where $g_{i,e}$ is the router's combining weight and f_e is expert e. When different selected experts live on different ranks, the dispatch is commonly an all-to-all exchange: every rank may send a different token subset to every other rank. A second exchange returns the computed outputs. GShard demonstrated conditional expert computation and large-scale automatic sharding, illustrating why routing and placement must be considered together [5].

Expert Parallelism can make large sparse capacity feasible, but it introduces a load-balance problem that ordinary dense Tensor Parallelism does not have. A popular expert may receive too many tokens while others idle; an uneven request batch can leave the communication fabric busy even when the arithmetic per token is modest. Capacity limits, routing policies, and token distribution therefore affect both quality behavior and serving latency. In an interactive setting, the slowest required expert route can determine when a token is ready.

6 Communication on the Serving Critical Path

Distributed inference communicates several different tensor relationships. Naming the operation makes the ownership change explicit and helps expose when a plan can overlap communication with useful work.

Operation	Resulting relationship	Common inference use
All-Reduce	Reduce a tensor across a group and deliver the result to every rank.	Sum row-parallel partial outputs or synchronize a replicated intermediate.
All-Gather	Collect shards so that each rank receives a full logical tensor.	Materialize a feature or parameter view needed by the next operation.
Reduce-Scatter	Reduce a tensor and leave a different shard on each rank.	Return from a replicated intermediate to a sharded activation layout.
Point-to-point	Send one activation or state object from one rank to another.	Transfer activations between Pipeline stages or hand off explicit request state.
All-to-All	Each rank exchanges a distinct payload with every other rank.	Dispatch and return tokens for Expert Parallel MoE computation.

Table 1: Collectives describe which rank owns a result after communication. The appropriate operation follows the tensor partition, not a generic preference for one collective.

For a single communication event with payload q, a minimal cost model is

$$t_{\mathrm{comm}(q)} \approx \alpha + \frac{q}{\mathrm{BW}}, \tag{5}$$

where α summarizes startup and synchronization latency and BW is an effective bandwidth. Collective algorithms, topology, congestion, and message shape change both quantities, so Equation 5 is not a benchmark model. It explains the enduring distinction: frequent small events tend to be latency-sensitive, whereas long KV transfers or large activation exchanges can become bandwidth-sensitive.

For a request, an equally useful decomposition is

$$t_{\mathrm{request}} \approx t_{\mathrm{compute}} + t_{\mathrm{communication}} + t_{\mathrm{synchronization}} + t_{\mathrm{scheduling}}. \tag{6}$$

The terms may overlap, so the expression should not be read as an exact additive accounting. It is a diagnostic model. In particular, Decode repeats a short dependency chain, giving little time to hide a late collective. Prefill can often amortize communication over more tokens and more arithmetic, whereas Decode exposes latency at each output position. The system design should report both phase-specific measurements rather than averaging them into one opaque throughput number.

7 Prefill–Decode Disaggregation

Prefill and Decode need not use the same accelerators or the same parallel plan. A **Prefill–Decode disaggregated** service assigns prompt processing to one pool, constructs a request's KV Cache, and then transfers the cache to, or makes it accessible from, a Decode pool. The Decode pool continues the autoregressive loop and owns the active request afterward. The design aims to prevent long prompt computation from interfering with the short, latency-sensitive Decode iterations described in Chapters 19 and 22.

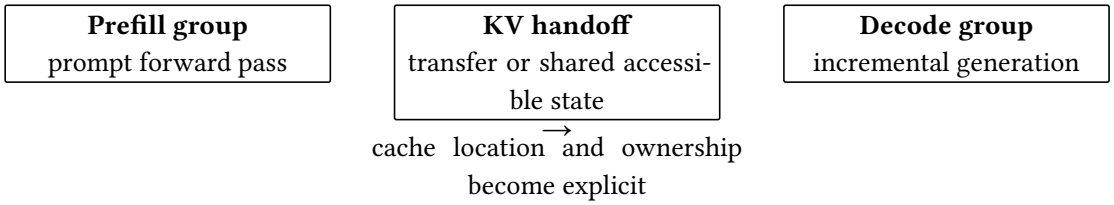

Figure 2: Disaggregation separates the phase that constructs KV state from the phase that repeatedly consumes and extends it. It removes neither cache-placement requirements nor the cost of making the correct state available to Decode.

The benefit is not automatic. Moving a large cache across a constrained link can consume enough bandwidth or delay to erase the queueing benefit. The receiving group must also validate the model revision, tokenizer-derived positions, attention layout, numerical representation, and exact committed prefix before using transferred entries. If cache blocks are shared rather than copied, the service still needs a lifetime, access, and reclamation protocol. DistServe studies the interference caused by colocating the two phases and optimizes their resource allocation and placement separately [6]. Its result motivates phase-aware planning; it does not imply that every workload benefits from a disaggregated design.

Disaggregation can also change parallelism choices. A high-throughput Prefill pool may favor larger batches or a plan that exploits prompt-level parallelism, while a Decode pool may prefer smaller communication groups and a layout tuned for per-token responsiveness. The two pools must agree on a portable cache representation or carry enough metadata to transform it correctly. Incompatible KV head layouts, positional conventions, quantization formats, or partition ownership rules are correctness errors, not merely performance issues.

8 Composing Parallelism Dimensions

No dimension is inherently a complete serving strategy. Tensor Parallelism addresses the width and memory of individual layers; Pipeline Parallelism addresses depth placement; Expert Parallelism addresses sparse expert placement; replicas provide request-level capacity. A system may combine them as

$$W = n_R n_T n_P n_E, \tag{7}$$

where n_E is included only when Expert Parallel groups are an independent placement dimension. Sequence Parallelism is omitted from Equation 7 when it reuses the Tensor Parallel group. The formula describes resource ownership, not end-to-end speed: larger products can lower local memory while raising communication, queueing, or underutilization costs.

Observed constraint	Likely first response	Condition to verify
One layer or full model does not fit	Introduce a modest Tensor Parallel group; use Pipeline Parallelism if depth still cannot fit.	Weight and workspace savings exceed added collective and stage-hop latency.
One group fits but request arrival rate is too high	Add complete replicas and route requests with cache affinity.	Each replica has enough queue and cache capacity for its assigned workload.
Sparse experts dominate memory or compute	Place experts across an Expert Parallel group.	Routing balance and all-to-all traffic remain within the latency budget.
Long prompts interfere with interactive Decode	Evaluate phase-specific pools or controlled Prefill–Decode disaggregation.	KV handoff and network cost do not offset reduced phase interference.

Table 2: A parallelism plan should start from a measured resource constraint, then test the new communication and state-management costs under the intended workload.

Table 2 is intentionally conditional. For example, a model that fits on one accelerator may still benefit from replicas before it benefits from model parallelism, because replication avoids per-layer communication. Conversely, a model that cannot fit has no replica-only solution. AlpaServe highlights that model-parallel serving involves a trade-off between its overhead and the opportunity to multiplex bursty workloads across devices [7]. A sound plan maps tightly communicating ranks to favorable topology, keeps request-routing and state ownership explicit, and measures TTFT, TPOT, throughput, queueing delay, and cache utilization together.

9 Failure Modes and Scaling Limits

The limiting rank or link on a request's critical path determines the perceived result. Communication bottlenecks arise when collectives are too frequent, payloads are too large, or the chosen group spans an unsuitable network boundary. Pipeline bubbles and unbalanced stages leave devices idle, especially when the active batch is too small to fill the pipeline. MoE routing can create expert hotspots and all-to-all contention. A straggler rank, transient network fault, or collective-order mismatch can delay every rank in an execution group; the system must distinguish a recoverable slow request from a failed group rather than indefinitely holding cache capacity.

Serving adds failures that are less prominent in training. An admission policy may schedule a request on a computationally available replica whose KV Cache cannot grow to the requested context limit. A handoff can send cache blocks to a Decode group with a different model version or incompatible shard layout. A topology-unaware autoscaler can place Tensor Parallel peers across a slower boundary, changing per-token latency without changing the nominal device count. Insufficient batching can make pipeline or Tensor Parallel communication dominate, while aggressive batching can satisfy throughput measurements but violate interactive latency. These are placement and state-lifecycle failures, not weaknesses in the language-model objective.

10 Implementation Contracts

The **placement contract** must declare every execution group, replica, shard axis, Pipeline stage, Expert owner, and communication group. For each tensor that crosses a boundary, it must state the logical shape, layout, collective or point-to-point operation, numerical representation, and post-operation owner. A small deterministic prompt should produce logits matching a single-group reference within the stated numerical tolerance for each supported parallel layout. All ranks in a collective group must execute compatible collectives in the same order; an inference runtime cannot recover correctness by allowing only some ranks to advance.

The **request-state contract** must bind a request identifier to its tokenizer and model revision, committed token prefix, positions, sampling configuration, KV block locations, cache format, and owning execution group. A migration or Prefill–Decode handoff must be explicit, atomic from the request's point of view, and validated before Decode uses the state. Cancellation, completion, preemption, and

failure recovery must reclaim every cache allocation exactly once. The contract must state whether state is copied, remotely accessible, or recomputed and how it remains valid while a request is active. The **observability contract** should report phase-specific TTFT and TPOT, tokens per second, queueing delay, per-group batch occupancy, cache capacity and allocation failures, collective wait time, inter-stage transfer time, expert-load distribution, and retry or restart events. Aggregate throughput cannot diagnose an execution group stalled on one all-reduce or a cache handoff slowed by a network link. Deployment comparisons should preserve model revision, cache representation, request mix, context and output limits, sampling policy, and service objective, so that a claimed parallelism gain refers to the same serving behavior.

11 Summary

Distributed inference has two complementary purposes: fitting one request execution across multiple accelerators and serving more independent requests. Tensor Parallelism splits layer computation and introduces frequent collectives; Pipeline Parallelism splits depth and introduces stage-to-stage dependencies; Expert Parallelism routes sparse computation and can require all-to-all exchange. Replicas, by contrast, are complete execution groups that add request capacity while preserving cache affinity. The cost of this placement is communication and state management on the serving critical path. Prefill can often amortize larger operations, while Decode repeatedly exposes synchronization and link latency. Prefill–Decode disaggregation can reduce phase interference only when KV state can be transferred or made available safely and efficiently. A useful deployment therefore chooses its parallel dimensions from measured capacity, topology, cache, and service constraints, then verifies that its placement contracts preserve both the model's logits and the request's state throughout generation.

References

[1] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[2] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019, doi: 10.48550/arXiv.1909.08053.

[3] Y. Huang *et al.*, "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019. doi: 10.48550/arXiv.1811.06965.

[4] D. Narayanan *et al.*, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021. doi: 10.1145/3458817.3476209.

[5] D. Lepikhin *et al.*, "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," *arXiv preprint arXiv:2006.16668*, 2020, doi: 10.48550/arXiv.2006.16668.

[6] Y. Zhong *et al.*, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[7] Z. Li *et al.*, "AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving," in *17th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2023, pp. 663–679. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/li-zhouhan